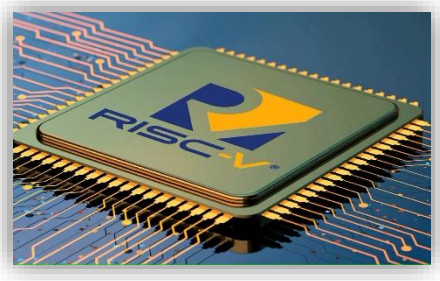

Subject: Digital Electronics

Topic: RISC-V processor

By:

Merna Nagy Farah Hana

Mariam Gomaa Khalaf Mostafa

1_Main Modules

1.1 ALU

The Arithmetic Logic Unit (ALU) is designed to perform a set of fundamental operations required by the processor, including addition, subtraction, logical shifts, and bitwise operations. Based on a 3-bit control signal, the ALU computes the result of the selected operation and provides two status flags: Zero, which indicates if the result is zero, and Sign as shown in fig1 and fig2

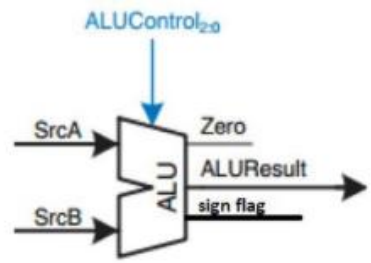

Fig1.inputs and outputs of ALU

moduleALU.v

```
module Alu (
    input [31:0] SrcA, SrcB,
    input [2:0] ALUControl,
    output reg [31:0] ALUResult,
    output Zero,
    output Sign
);

always @(*) begin
    case (ALUControl)
        3'b000: ALUResult = SrcA + SrcB;
        3'b001: ALUResult = SrcA << SrcB[4:0];
        3'b010: ALUResult = SrcA - SrcB;
        3'b100: ALUResult = SrcA ^ SrcB;
        3'b101: ALUResult = SrcA >> SrcB[4:0];
        3'b110: ALUResult = SrcA | SrcB;
        3'b111: ALUResult = SrcA & SrcB;
        default: ALUResult = 32'b0;
    endcase
end

assign Zero = (ALUResult == 32'b0);
assign Sign = ALUResult[31];

endmodule
```

Fig2.the implementation of ALU module

1.2 Program Counter

The **Program Counter (PC)** module calculates the address of the next instruction to be executed, either by incrementing the current PC by 4 or by adding an immediate offset for control flow changes (branches). It updates the PC value on the rising clock edge if load is asserted, and resets to zero when reset is low as shown in fig4.

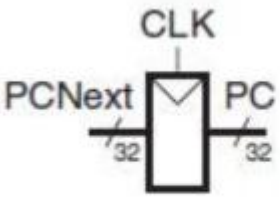

Fig3.programme counter module

PC_module.v

```
module ProgramCounter(
    input [31:0] ImmExt,
    input CLK, areset, load, PCSrc,
    output reg [31:0] PC
);
wire [31:0] PCNext = (PCSrc) ? (PC + ImmExt) : (PC + 4);

always @(posedge CLK or negedge areset) begin
    if (!areset)
        PC <= 32'b0;
    else if (load)
        PC <= PCNext;
end

endmodule
```

Fig4.the implementation of PC module

1.3 Instruction memory

The Instruction Memory module stores the program instructions and provides a 32-bit instruction output (RD) based on the input address (A). It loads the instruction contents from an external file (program.txt).

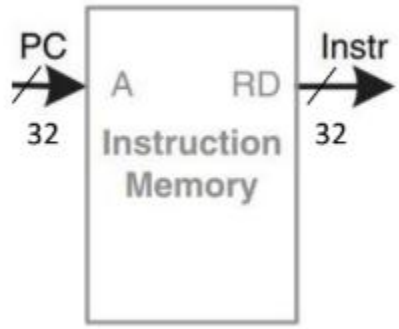

Fig5.instruction memory module

Instr_mem_module.v

```
module InstructionMemory (
    input [31:0] A,
    output [31:0] RD
);

reg [31:0] mem [0:63];

initial begin
    $readmemh("program.txt", mem);
end

assign RD = mem[A[31:2]];

endmodule
```

Fig6.the implementation of instruction memory module

1.4 Register File

The **Register File** module provides access to 32 registers used for instruction execution, supporting two read ports (RD1, RD2) and one write port. On each clock cycle, if WE3 is enabled and the destination register (A3) is not zero, data is written; all registers are reset to zero when areset is lows shown in fig8.

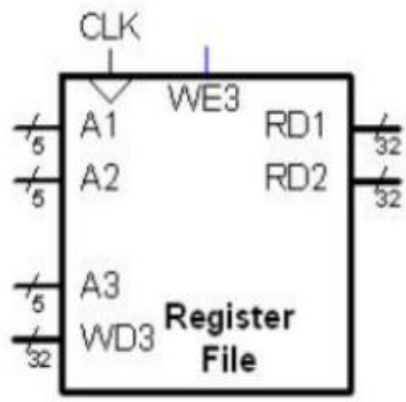

Fig7.register file module

Register_File.v

```
module Register_File(
    input CLK, areset, WE3,
    input [4:0] A1, A2, A3,
    input [31:0] WD3,
    output [31:0] RD1, RD2
);

reg [31:0] regs[0:31];
integer i;

always @(posedge CLK or negedge areset) begin
    if (!areset) begin
        for (i = 0; i < 32; i = i + 1)
            regs[i] <= 32'b0;
    end else if (WE3 && A3 != 5'b0) begin
        regs[A3] <= WD3;
    end
end

assign RD1 = (A1 == 5'b0) ? 32'b0 : regs[A1];
assign RD2 = (A2 == 5'b0) ? 32'b0 : regs[A2];

endmodule
```

Fig8.the implementation of register file module

1.5 Data Memory

The **Single-Port RAM** module simulates data memory for the processor, allowing word-aligned read and write operations through a single shared address port. Writing operations occur on the rising clock edge when WE is enabled, while readings are performed asynchronously using the 32-bit address input as shown in fig10.

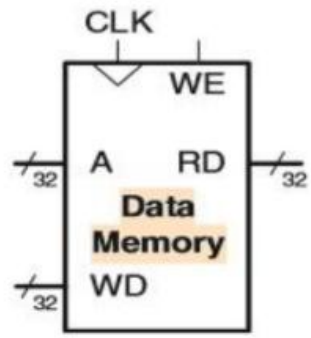

Fig8.data memory module

moduleDataMemory.v

```
module single_port_ram
(
input CLK,WE,
input [31:0] A,
input [31:0] WD,
output [31:0] RD
);
reg [31:0] mem [0:63]; //memory declartion
always @(posedge CLK) begin
if (WE)
    mem[A[31:2]] <= WD;
end
// Asynchronous read operation
assign RD = mem[A[31:2]];
endmodule
```

Fig10.the implementation of data memory module

1.6 Control Unit

1.6.1 Main Decoder

Generates control signals based on the opcode to manage register writes, memory access, immediate types, and ALU operation type. Supports instruction types like R-type, I-type, and branches

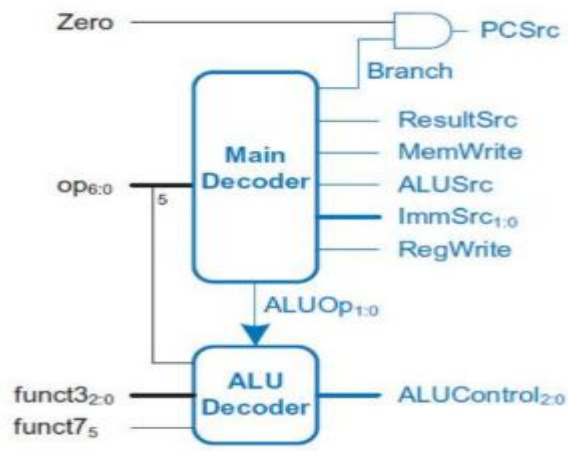

Fig8.control unit module s

```
module.v
module ControlUnit(
    input [6:0] op,
    input [2:0] funct3,
    input funct7, op5, Zero, SignFlag,
    output reg RegWrite, ALUSrc, MemWrite, Branch, ResultSrc, PCSrc,
    output reg [1:0] ImmSrc,
    output reg [2:0] ALUControl
);

reg [1:0] ALUOp;

// Main Decoder
always @(*) begin
    case (op)
        7'b0000011: begin // lw
            RegWrite = 1;
            ImmSrc = 2'b00;
            ALUSrc = 1;
            MemWrite = 0;
            ResultSrc = 1;
            Branch = 0;
            ALUOp = 2'b00;
        end
        7'b0100011: begin // sw
            RegWrite = 0;
            ImmSrc = 2'b01;
            ALUSrc = 1;
            MemWrite = 1;
            ResultSrc = 0;
            Branch = 0;
            ALUOp = 2'b00;
        end
        7'b0110011: begin // R-type
            RegWrite = 1;
            ImmSrc = 2'b00;
            ALUSrc = 0;
            MemWrite = 0;
            ResultSrc = 0;
            Branch = 0;
            ALUOp = 2'b10;
        end
        7'b0010011: begin // I-type
            RegWrite = 1;
            ImmSrc = 2'b00;
            ALUSrc = 1;
            MemWrite = 0;
            ResultSrc = 0;
            Branch = 0;
            ALUOp = 2'b10;
        end
        7'b1100011: begin // Branch
            RegWrite = 0;
            ImmSrc = 2'b10;
            ALUSrc = 0;
            MemWrite = 0;
            ResultSrc = 0;
            Branch = 1;
            ALUOp = 2'b01;
        end
        default: begin
            RegWrite = 0;
            ImmSrc = 2'b00;
            ALUSrc = 0;
            MemWrite = 0;
            ResultSrc = 0;
            Branch = 0;
            ALUOp = 2'b00;
        end
    endcase
end
```

Fig11.the implementation of main decoder

1.6.2 ALU Decoder and PCSrc Logic

ALU Decoder

Determines the specific ALU operation (add, sub, xor) using ALUOp, funct3, funct7, and op[5], enabling correct execution of arithmetic and logical instructions.

PCSrc Logic

Controls branching by setting PCSrc based on the instruction type (funct3) and condition flags (Zero, SignFlag)

```
// PCSrc Logic
always @(*) begin
    case (funct3)
        3'b000: PCSrc = Zero & Branch;       // beq
        3'b001: PCSrc = ~Zero & Branch;      // bne
        3'b100: PCSrc = SignFlag & Branch;   // blt
        default: PCSrc = 0;
    endcase
end

// ALU Decoder
always @(*) begin
    case (ALUOp)
        2'b00: ALUControl = 3'b000; // add (for lw/sw)
        2'b01: ALUControl = 3'b010; // subtract (for branches)
        2'b10: begin
            case (funct3)
                3'b000: ALUControl = ({op5, funct7} == 2'b11) ? 3'b010 : 3'b000; // sub/add
                3'b001: ALUControl = 3'b001; // sll
                3'b100: ALUControl = 3'b100; // xor
                3'b101: ALUControl = 3'b101; // srl
                3'b110: ALUControl = 3'b110; // or
                3'b111: ALUControl = 3'b111; // and
                default: ALUControl = 3'b000;
            endcase
        end
        default: ALUControl = 3'b000;
    endcase
end

endmodule
```

Fig12.the implementation of Alu decoder

1.7 small modules

1.7.1 Sign extend

moduleSignextend.v

```
module signextend (
    input [31:0] Instr,
    input [1:0] ImmSrc,
    output reg [31:0] ImmExt
);

always @(*) begin
    case (ImmSrc)
        2'b00: ImmExt = {{20{Instr[31]}}, Instr[31:20]};                          // I-type
        2'b01: ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]};             // S-type
        2'b10: ImmExt = {{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}; // B-type
        default: ImmExt = 32'b0;
    endcase
end

endmodule
```

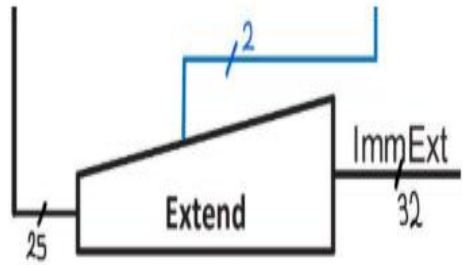

Fig13.the implementation of sign extention

1.7.2 MUX 2x1

mux.v

```
module MUX2to1 (
    input [31:0] In1, In2,
    input sel,
    output [31:0] out
);
assign out = sel ? In2 : In1;

endmodule
```

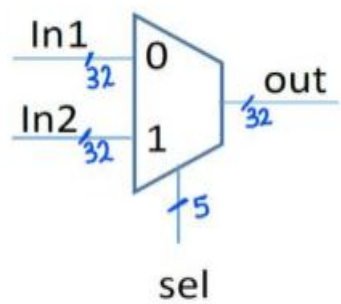

Fig14.the implementation of 2X1 MUX

2_TOP module

The **Top** module integrates all components of a simplified RISC-V processor, including the program counter, instruction memory, register file, ALU, control unit, and data memory. It coordinates instruction fetching, decoding, execution, memory access, and write-back using control signals to simulate a basic single-cycle RISC-V CPU as shown in fig15.

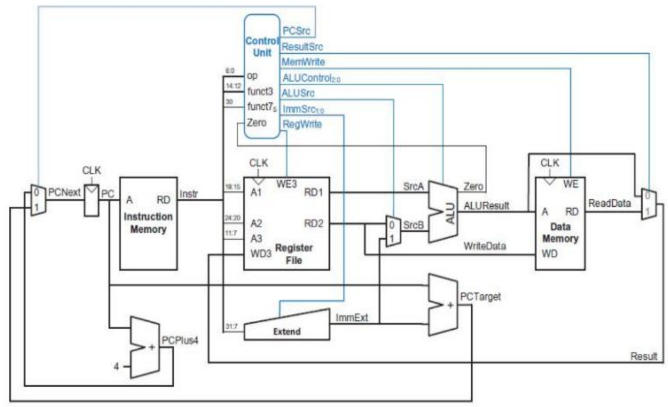

Fig15. Complete single-cycle RISC-V

```
dule RV_32I.v
module RISCV_Processor_Top (
    input CLK,
    input areset
);

// Internal signals
wire [31:0] PC;
wire [31:0] Instr;
wire [31:0] RD1, RD2;
wire [31:0] ImmExt;
wire [31:0] SrcA, SrcB;
wire [31:0] ALUResult;
wire [31:0] ReadData;
wire [31:0] Result;

// Control signals
wire RegWrite, ALUSrc, MemWrite, ResultSrc, PCSrc, Branch;
wire [1:0] ImmSrc;
wire [2:0] ALUControl;
wire Zero, SignFlag;

// Program Counter
ProgramCounter pc_unit (
    .ImmExt(ImmExt),
    .CLK(CLK),
    .areset(areset),
    .load(1'b1),  // always enabled, no halt
    .PCSrc(PCSrc),
    .PC(PC)
);

// Instruction Memory
InstructionMemory imem (
    .A(PC),
    .RD(Instr)
);

// Register File
Register_File regfile (
    .CLK(CLK),
    .areset(areset),
    .WE3(RegWrite),
    .A1(Instr[19:15]), // rs1
    .A2(Instr[24:20]), // rs2
    .A3(Instr[11:7]),  // rd
    .WD3(Result),
    .RD1(RD1),
    .RD2(RD2)
);
```

```
// Sign Extend
signextend signext (
    .Instr(Instr),
    .ImmSrc(ImmSrc),
    .ImmExt(ImmExt)
);
// ALU Source A
assign SrcA = RD1;
// ALU Source B MUX
MUX2to1 alu_src_mux (
    .In1(RD2),      // 0: register
    .In2(ImmExt),   // 1: immediate
    .sel(ALUSrc),
    .out(SrcB)
);
// ALU
Alu alu (
    .SrcA(SrcA),
    .SrcB(SrcB),
    .ALUControl(ALUControl),
    .ALUResult(ALUResult),
    .Zero(Zero),
    .Sign(SignFlag)
);
// Data Memory
single_port_ram dmem (
    .CLK(CLK),
    .WE(MemWrite),
    .A(ALUResult),
    .WD(RD2),
    .RD(ReadData)
);
// Result MUX
MUX2to1 result_mux (
    .In1(ALUResult),   // 0: from ALU
    .In2(ReadData),    // 1: from memory
    .sel(ResultSrc),
    .out(Result)
);
// Control Unit
ControlUnit control (
    .op(Instr[6:0]),
    .funct3(Instr[14:12]),
    .funct7(Instr[30]),
    .op5(Instr[5]),
    .Zero(Zero),
    .SignFlag(SignFlag),
    .RegWrite(RegWrite),
    .ALUSrc(ALUSrc),
    .MemWrite(MemWrite),
    .Branch(Branch),        // now connected
    .ResultSrc(ResultSrc),
    .PCSrc(PCSrc),
    .ImmSrc(ImmSrc),
    .ALUControl(ALUControl)
```

Fig16.the implementation of single-cycle RISC-V

3_Result

By trying to execute the instructions in the programme.txt to implement the The **Fibonacci sequence** on modalism we found the output as shown in figure 17 and 18.

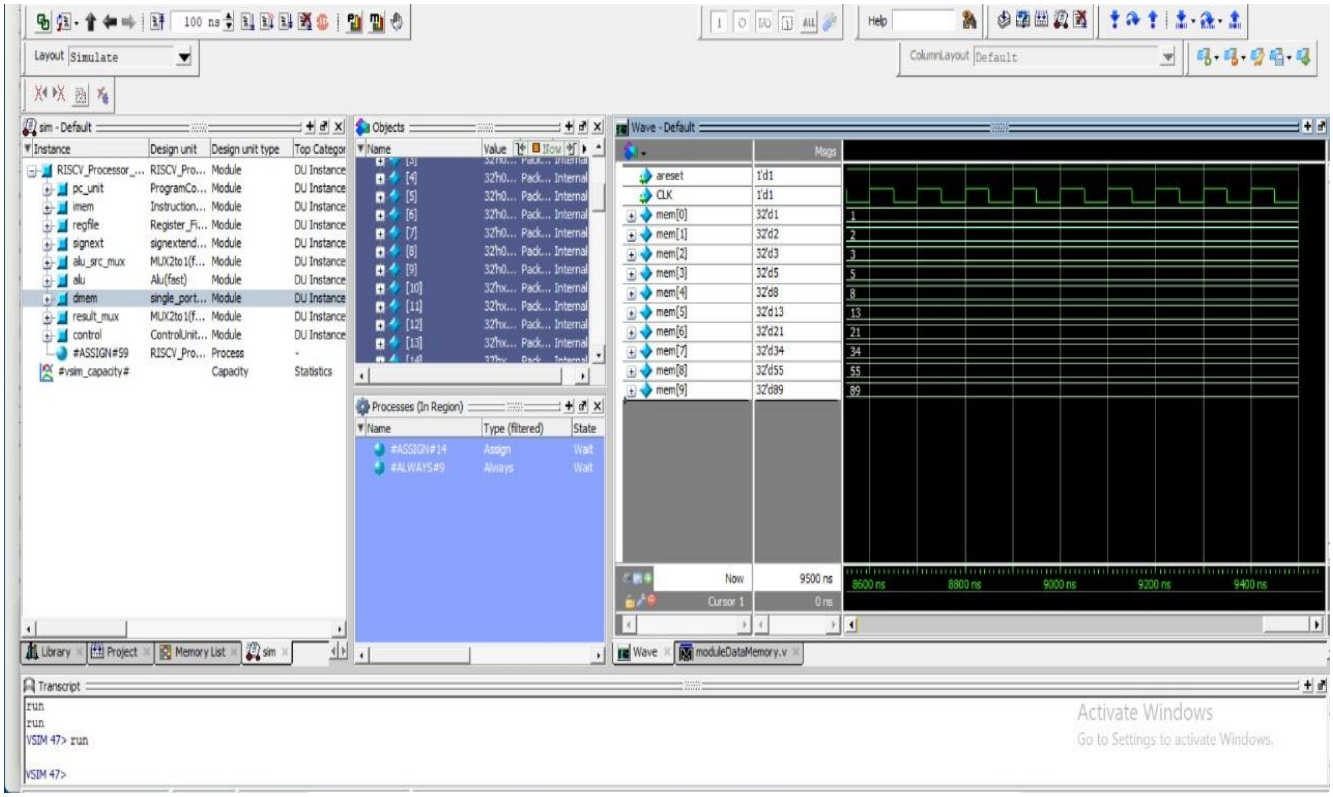

Fig17.the simulation window of top module

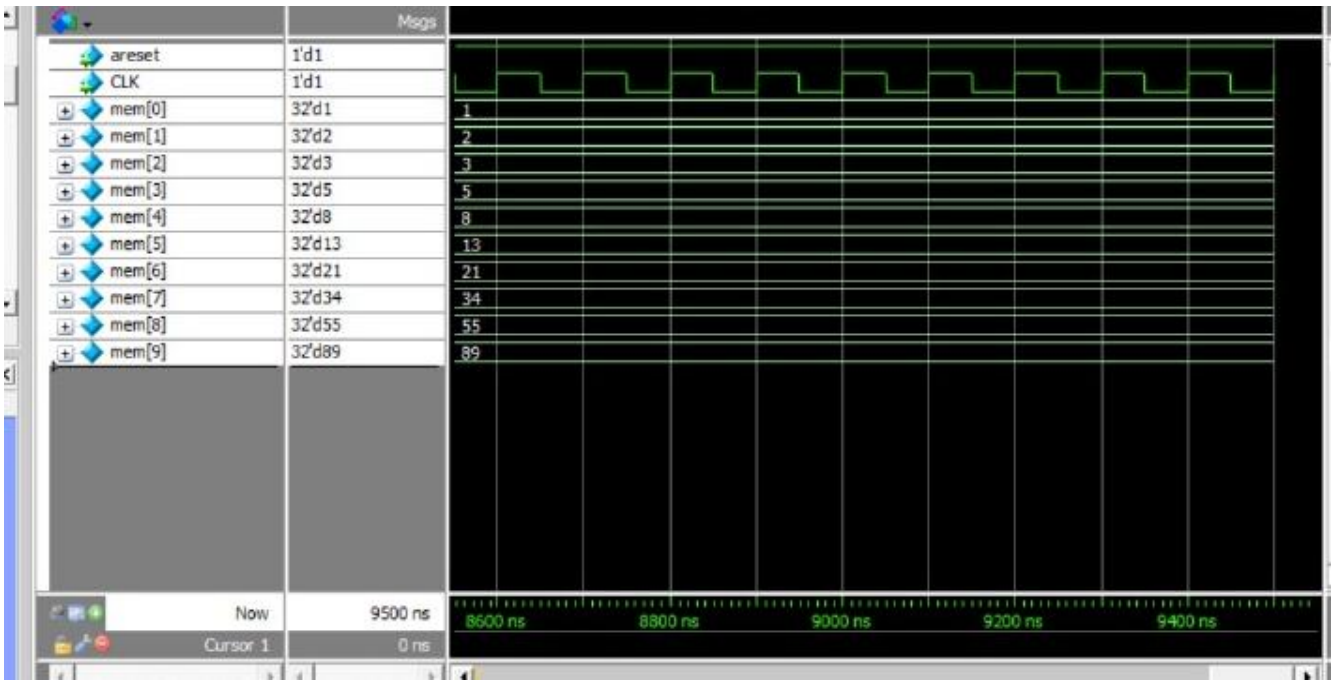

Fig18. The Fibonacci sequence